Homework Set-8

Roll Number: 2025115140

1. BFS and DFS

```
#include <stdio.h>
#define N 7
int graph[N][N];
int visited[N];
char label[N] = {'A','B','C','E','F','G','H'};
void DFS(int v)
{
    int i;
    visited[v] = 1;
    printf("%c ", label[v]);
    for (i = 0; i < N; i++)
        if (graph[v][i] && !visited[i])
            DFS(i);
}
void BFS(int start)
{
    int queue[N];
    int front = 0, rear = 0;
    int i, v;
    for (i = 0; i < N; i++)
        visited[i] = 0;
    queue[rear++] = start;
    visited[start] = 1;
    while (front < rear)
    {
        v = queue[front++];
        printf("%c ", label[v]);
        for (i = 0; i < N; i++)
            if (graph[v][i] && !visited[i])
            {
                visited[i] = 1;
                queue[rear++] = i;
            }
    }
}
int main()
```

```
{
    int edges[][2] = {
        {0,1}, {0,5}, {0,6}, {1,3},
        {3,5}, {1,4}, {4,6}, {4,2}
    };
    int i;
    for (i = 0; i < 8; i++)
    {
        graph[edges[i][0]][edges[i][1]] = 1;
        graph[edges[i][1]][edges[i][0]] = 1;
    }
    printf("BFS (starting from A): ");
    BFS(0);
    for (i = 0; i < N; i++)
        visited[i] = 0;
    printf("\nDFS (starting from A): ");
    DFS(0);
    return 0;
}
```

Output:

```
BFS (starting from A): A B G H E F C
DFS (starting from A): A B E G F C H
```

2. Minimum Spanning Tree — Prim's and Kruskal's

```
#include <stdio.h>
#define INF 999
#define N 5
int graph[N][N] = {
    {0,1,7,0,0},
    {1,0,5,4,3},
    {7,5,0,0,6},
    {0,4,0,0,2},
    {0,3,6,2,0}
};
void prim()
{
    int selected[N] = {1,0,0,0,0};
    int edges = 0, total = 0;
    int i, j, x = -1, y = -1, min;
    printf("Prim's MST:\n");
    while (edges < N - 1)
    {
        min = INF;
        for (i = 0; i < N; i++)
            if (selected[i])
                for (j = 0; j < N; j++)
                    if (!selected[j] && graph[i][j] && graph[i][j] < min)
                    {
                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
        printf("%c - %c = %d\n", 'A'+x, 'A'+y, min);
        selected[y] = 1;
        total += min;
        edges++;
    }
    printf("Prim cost = %d\n", total);
}
int parent[N];
int find(int x)
{
    while (parent[x] != x)
        x = parent[x];
    return x;
```

```
}
void unite(int a, int b)
{
    parent[find(a)] = find(b);
}
struct Edge { int u, v, w; };
void kruskal()
{
    struct Edge e[] = {
        {0,1,1},{0,2,7},{1,2,5},{1,3,4},
        {1,4,3},{2,4,6},{3,4,2}
    };
    int m = 7, i, j, total = 0, count = 0;
    struct Edge temp;
    for (i = 0; i < N; i++)
        parent[i] = i;
    for (i = 0; i < m-1; i++)
        for (j = 0; j < m-i-1; j++)
            if (e[j].w > e[j+1].w)
            {
                temp = e[j];
                e[j] = e[j+1];
                e[j+1] = temp;
            }
    printf("Kruskal's MST:\n");
    for (i = 0; i < m && count < N-1; i++)
        if (find(e[i].u) != find(e[i].v))
        {
            printf("%c - %c = %d\n", 'A'+e[i].u, 'A'+e[i].v, e[i].w);
            total += e[i].w;
            unite(e[i].u, e[i].v);
            count++;
        }
    printf("Kruskal cost = %d\n", total);
}
int main()
{
    prim();
    kruskal();
    return 0;
}
```

Output:

```
Prim's MST:
A - B = 1
B - E = 3
E - D = 2
B - C = 5
Prim cost = 11
Kruskal's MST:
A - B = 1
D - E = 2
B - E = 3
B - C = 5
Kruskal cost = 11
```

3. Minimum Spanning Tree — Kruskal's Algorithm

```
#include <stdio.h>
#define V 9
#define MAXV 10
struct Edge { int u, v, w; };
int parent[MAXV];
int find(int x)
{
    while (parent[x] != x)
        x = parent[x];
    return x;
}
void unite(int a, int b)
{
    parent[find(a)] = find(b);
}
int main()
{
    struct Edge e[] = {
        {5,7,2},{3,6,5},{1,4,6},{6,8,6},
        {3,4,7},{4,7,7},{2,5,8},{7,9,8},
        {1,3,9},{1,2,10},{8,9,10},{1,5,12},
        {5,9,13},{6,7,14}
    };
    int m = 14, i, j, count = 0, total = 0;
    struct Edge temp;
    for (i = 1; i <= V; i++)
        parent[i] = i;
    for (i = 0; i < m-1; i++)
        for (j = 0; j < m-i-1; j++)
            if (e[j].w > e[j+1].w)
            {
                temp = e[j];
                e[j] = e[j+1];
                e[j+1] = temp;
            }
    printf("Edges in MST:\n");
    for (i = 0; i < m && count < V-1; i++)
        if (find(e[i].u) != find(e[i].v))
        {
            printf("%c - %c = %d\n", 'A'+e[i].u, 'A'+e[i].v, e[i].w);
            total += e[i].w;
```

```
            unite(e[i].u, e[i].v);
            count++;
        }
    printf("Minimum cost = %d\n", total);
    return 0;
}
```

Output:

```
Edges in MST:
F - H = 2
D - G = 5
B - E = 6
G - I = 6
D - E = 7
E - H = 7
C - F = 8
H - J = 8
Minimum cost = 49
```

4. Dijkstra's Algorithm

```
#include <stdio.h>
#define V 8
#define INF 999
int main()
{
    int g[7][7] = {
        {0},
        {0,0,1,1,0,0,0},
        {0,0,0,0,1,1,0},
        {0,0,0,0,1,0,1},
        {0,0,0,0,0,0,1},
        {0,0,0,0,0,0,1},
        {0,0,0,0,0,0,0}
    };
    int dist[V], visited[V] = {0};
    int i, j, u = -1, min;
    for (i = 0; i < V; i++)
        dist[i] = INF;
    dist[0] = 0;
    for (i = 0; i < V-1; i++)
    {
        min = INF;
        for (j = 0; j < V; j++)
            if (!visited[j] && dist[j] < min)
            {
                min = dist[j];
                u = j;
            }
        visited[u] = 1;
        for (j = 0; j < V; j++)
            if (g[u][j] && !visited[j] &&
                dist[u] + g[u][j] < dist[j])
                dist[j] = dist[u] + g[u][j];
    }
    printf("Shortest distances from A:\n");
    for (i = 0; i < V; i++)
        printf("%d = %d\n", i, dist[i]);
    return 0;
}
```

Output:

```
Shortest distances from A:
0 = 0
1 = 999
2 = 999
3 = 999
4 = 999
5 = 999
6 = 999
7 = 999
```

5. Topological Sorting

```
#include <stdio.h>
#define V 6
int main()
{
    int g[7][7] = {
        {0},
        {0,0,1,1,0,0,0},
        {0,0,0,0,1,1,0},
        {0,0,0,0,1,0,1},
        {0,0,0,0,0,0,1},
        {0,0,0,0,0,0,1},
        {0,0,0,0,0,0,0}
    };
    int indegree[7] = {0};
    int queue[7], front=0, rear=0;
    int i,j,v;
    for(i=1;i<=V;i++)
        for(j=1;j<=V;j++)
            if(g[i][j])
                indegree[j]++;
    for(i=1;i<=V;i++)
        if(indegree[i]==0)
            queue[rear++]=i;
    printf("Topological order: ");
    while(front<rear)
    {
        v=queue[front++];
        printf("%d ",v);
        for(j=1;j<=V;j++)
            if(g[v][j] && --indegree[j]==0)
                queue[rear++]=j;
    }
    return 0;
}
```

Output:

```
Topological order: 1 2 3 5 4 6
```

6. Linear Probing

```
#include <stdio.h>
#define SIZE 7
int main()
{
    int table[SIZE];
    int keys[] = {76,93,40,47,10,55};
    int i, j, index;
    for (i = 0; i < SIZE; i++)
        table[i] = -1;
    for (i = 0; i < 6; i++)
    {
        index = keys[i] % SIZE;
        j = 0;
        while (table[(index+j) % SIZE] != -1)
            j++;
        table[(index+j) % SIZE] = keys[i];
    }
    printf("Final hash table:\n");
    for (i = 0; i < SIZE; i++)
        printf("%d : %d\n", i, table[i]);
    return 0;
}
```

Output:

```
Final hash table:
0 : 47
1 : 55
2 : 93
3 : 10
4 : -1
5 : 40
6 : 76
```

7. Quadratic Probing

```
#include <stdio.h>
#define SIZE 7
int main()
{
    int table[SIZE];
    int keys[] = {22,30,50};
    int i, j, index;
    for (i = 0; i < SIZE; i++)
        table[i] = -1;
    for (i = 0; i < 3; i++)
    {
        index = keys[i] % SIZE;
        j = 0;
        while (table[(index+j*j) % SIZE] != -1)
            j++;
        table[(index+j*j) % SIZE] = keys[i];
    }
    printf("Final hash table:\n");
    for (i = 0; i < SIZE; i++)
        printf("%d : %d\n", i, table[i]);
    return 0;
}
```

Output:

```
Final hash table:
0 : -1
1 : 22
2 : 30
3 : -1
4 : -1
5 : 50
6 : -1
```

8. Double Hashing

```
#include <stdio.h>
#define SIZE 7
int main()
{
    int table[SIZE];
    int keys[] = {27,43,69,2,72};
    int i, j, h1, h2, index;
    for (i = 0; i < SIZE; i++)
        table[i] = -1;
    for (i = 0; i < 5; i++)
    {
        h1 = keys[i] % SIZE;
        h2 = 1 + (keys[i] % 5);
        j = 0;
        while (table[(h1+j*h2) % SIZE] != -1)
            j++;
        index = (h1+j*h2) % SIZE;
        table[index] = keys[i];
    }
    printf("Final hash table:\n");
    for (i = 0; i < SIZE; i++)
        printf("%d : %d\n", i, table[i]);
    return 0;
}
```

Output:

```
Final hash table:
0 : -1
1 : 43
2 : 2
3 : -1
4 : 69
5 : 72
6 : 27
```

9. Quadratic Probing — Proof

For quadratic probing, the probe position is $h(k,i) = (h(k) + i^2) \bmod m$. If two probe positions are equal, then $i^2 \equiv j^2 \pmod m$, so $(i-j)(i+j) \equiv 0 \pmod m$. For a prime table size m, the first $(m+1)/2$ probe positions are distinct. Therefore, if at least half of the table is empty, an empty position will be found. Hence a new element can always be inserted. Thus proved.

10. Separate Chaining

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 7
struct Node
{
    int key;
    struct Node *next;
};
struct Node *table[SIZE];
void initialize()
{
    int i;
    for (i = 0; i < SIZE; i++)
        table[i] = NULL;
}
void insert(int key)
{
    int index = key % SIZE;
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->key = key;
    newNode->next = table[index];
    table[index] = newNode;
}
int search(int key)
{
    int index = key % SIZE;
    struct Node *p = table[index];
    while (p != NULL)
    {
        if (p->key == key)
            return 1;
        p = p->next;
    }
    return 0;
}
void display()
{
    int i;
    struct Node *p;
    for (i = 0; i < SIZE; i++)
```

```
    {
        printf("%d : ", i);
        p = table[i];
        while (p != NULL)
        {
            printf("%d -> ", p->key);
            p = p->next;
        }
        printf("NULL\n");
    }
}
int main()
{
    initialize();
    insert(10);
    insert(17);
    insert(24);
    insert(31);
    display();
    printf("Search 24: %s\n",
           search(24) ? "Found" : "Not Found");
    printf("Search 50: %s\n",
           search(50) ? "Found" : "Not Found");
    return 0;
}
```

Output:

```
0 : NULL
1 : NULL
2 : NULL
3 : 31 -> 24 -> 17 -> 10 -> NULL
4 : NULL
5 : NULL
6 : NULL
Search 24: Found
Search 50: Not Found
```